Reporte Técnico de Actividades Práctico-Experimentales Nro. 06

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Alejandro Padilla
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	06
Tipo	Individual
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	Ing. José O. Guamán Q.
Fecha	2026-11-06
Horario	Jueves 10h30 a 11h30; Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-cuadrado y Kolmogorov-Smirnov, para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en el intervalo [0, 1].

Los objetivos específicos son:

1. Generar secuencias de números pseudoaleatorios utilizando el método congruencial lineal (LCG).
2. Aplicar la prueba estadística Chi-cuadrado para evaluar la uniformidad de las frecuencias observadas.
3. Aplicar la prueba Kolmogorov-Smirnov para comparar la distribución empírica con la distribución uniforme teórica.
4. Interpretar los estadísticos de prueba y sus respectivos valores p.
5. Comparar el comportamiento estadístico de diferentes configuraciones del generador pseudoaleatorio.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador con Linux.
- Python 3.14, con Matplotlib para las gráficas, y pytest y SciPy para las pruebas del código.
- Java/JDK 21.
- Visual Studio Code como entorno de desarrollo.
- Generador de números pseudoaleatorios: un LCG propio, el mismo en ambos lenguajes.

No se usaron NumPy ni Pandas. Los cálculos se programaron con la biblioteca estándar para poder repetirlos línea por línea en Java.

4. Procedimiento / Metodología Ejecutada

4.1. Hipótesis y criterio de decisión

Se contrastaron las hipótesis de la guía:

H0: los números generados siguen una distribución uniforme U(0, 1).

H1: los números generados no siguen una distribución uniforme U(0, 1).

El nivel de significancia es α = 0.05. Si p ≥ 0.05, no se rechaza la hipótesis nula; si p < 0.05, se rechaza.

4.2. Generador congruencial lineal

Cada valor se obtiene del anterior y se normaliza al intervalo [0, 1):

$$X_{n+1} = (a \cdot X_n + c) \text{ mód } m$$

$$U_n = \frac{X_n}{m}$$

Se generaron 10 000 números para cada configuración y, para la tabla de tamaños, también 1 000, 100 000 y 1 000 000. Se compararon cuatro configuraciones:

Tabla 1. Parámetros de las configuraciones del generador

Configuración	a	c	m	X_0
glibc	$1\,103\,515\,245$	$12\,345$	2^{32}	1
APE 5	$1\,664\,525$	$1\,013\,904\,223$	2^{32}	$12\,345$
RANDU	$65\,539$	0	2^{31}	1
Módulo pequeño	5	3	16	7

RANDU es un generador histórico de IBM, conocido por sus defectos. El módulo pequeño solo puede producir 16 valores distintos.

4.3. Prueba Chi-cuadrado

El intervalo [0, 1) se dividió en diez subintervalos de igual amplitud. La frecuencia esperada de cada uno es $E_i = N/10$ y el estadístico suma las desviaciones al cuadrado:

$$\chi^2 = \sum_{i=1}^{10} \frac{(O_i - E_i)^2}{E_i}$$

$$gl = 10 - 1 = 9$$

El valor p es la probabilidad de que una χ^2 con 9 grados de libertad supere el estadístico obtenido.

4.4. Prueba Kolmogorov-Smirnov

Los números se ordenaron de menor a mayor. En cada posición i se comparó la distribución acumulada empírica, i/N, con la teórica, $F(x) = x$:

$$D^+ = \max_{1 \le i \le N} \left(\frac{i}{N} - U_{(i)} \right)$$

$$D^- = \max_{1 \le i \le N} \left(U_{(i)} - \frac{i-1}{N} \right)$$

$$D = \max(D^+, D^-)$$

Aquí $U_{(i)}$ es el i-ésimo valor de la muestra ordenada.

El valor p se obtiene de la distribución de Kolmogorov.

4.5. Implementación en Python y en Java

El código de Python está organizado en modelo, vista y controlador (paquete simulacion_ape). El de Java sigue la misma división en el paquete ape6: model (Lcg, Distributions, UniformityTests), view (ConsoleView) y controller (SimulationController), con un Main que solo llama al controlador. Ambos usan las mismas fórmulas, en el mismo orden. Las bibliotecas estándar de Python y de Java no incluyen funciones para calcular los valores p, así que se programaron a mano en los dos lenguajes, con las mismas fórmulas.

Para Chi-cuadrado se usó la función gamma incompleta regularizada, $Q(gl/2,\ \chi^2/2)$. Para Kolmogorov-Smirnov se usó la serie de Kolmogorov con la corrección de Stephens:

$$\lambda = \left(\sqrt{N} + 0.12 + \frac{0.11}{\sqrt{N}}\right) D$$

Ambas se tomaron de Numerical Recipes [1]. SciPy se usó solo en las 26 pruebas de pytest, para contrastar esos valores. Este es el cálculo de D en Python:

```python
ordered = sorted(values)
n = len(ordered)
d_plus = 0.0
d_minus = 0.0
for i, u in enumerate(ordered, start=1):
    d_plus = max(d_plus, i / n - u)
    d_minus = max(d_minus, u - (i - 1) / n)
statistic = max(d_plus, d_minus)
```

Y este es un extracto del conteo de frecuencias y del estadístico Chi-cuadrado en Java, en la clase UniformityTests del modelo:

```java
for (double u : values) {
    observed[Math.min((int) (u * BINS), BINS - 1)]++;
}
double expected = (double) values.length / BINS;
double statistic = 0.0;
for (int o : observed) {
    double diff = o - expected;
    statistic += diff * diff / expected;
}
```

El tiempo de cada programa incluye la generación y las dos pruebas. Se midió tres veces y se tomó el menor, en ambos lenguajes.

5. Resultados

5.1. Secuencia generada

Los cinco primeros valores del generador glibc son:

Tabla 2. Primeros valores generados por el LCG glibc

n	X_n	U_n
1	$1\,103\,527\,590$	0.25693504
2	$2\,524\,885\,223$	0.58787065
3	$662\,824\,084$	0.15432576
4	$3\,295\,386\,429$	0.76726694
5	$4\,182\,499\,122$	0.97381396

5.2. Frecuencias del generador glibc

Con N = 10 000, la frecuencia esperada es 1000 por subintervalo. Las frecuencias observadas fueron:

Tabla 3. Frecuencias observadas y esperadas del generador glibc, N = 10 000

Subintervalo	**Observada** O_i	**Esperada** E_i
$[0.0,\ 0.1)$	1001	1000
$[0.1,\ 0.2)$	973	1000
$[0.2,\ 0.3)$	989	1000
$[0.3,\ 0.4)$	1015	1000
$[0.4,\ 0.5)$	1005	1000
$[0.5,\ 0.6)$	984	1000
$[0.6,\ 0.7)$	1017	1000
$[0.7,\ 0.8)$	985	1000
$[0.8,\ 0.9)$	1022	1000
$[0.9,\ 1.0)$	1009	1000

De ahí resultan $\chi^2 = 2.44$ con 9 grados de libertad y p = 0.983, así que no se rechaza H0. Kolmogorov-Smirnov da D = 0.0071 y p = 0.687. Los valores coinciden con los del ejemplo computacional de las diapositivas de la semana 6 ($\chi^2 = 2.44$, p = 0.983, D = 0.0071) [2]. Solo el valor p de Kolmogorov-Smirnov difiere, 0.687 frente a 0.686: aquí se usó la serie asintótica de Kolmogorov con la corrección de Stephens, que es una aproximación.

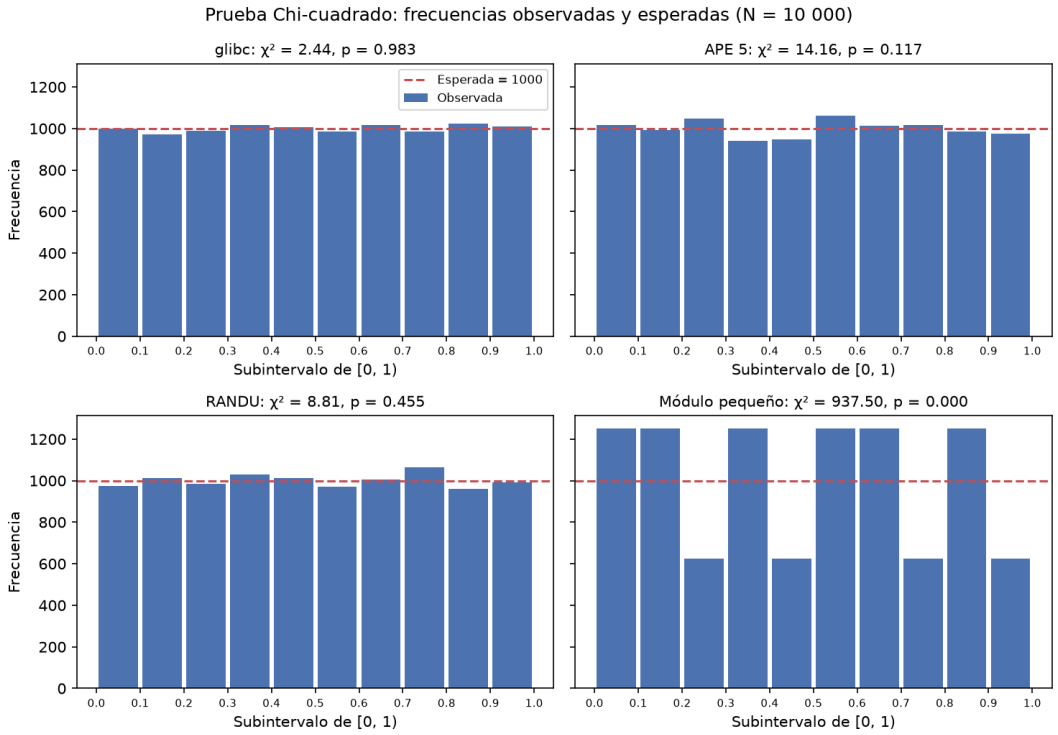


Figura 1: Frecuencias observadas y esperadas de las cuatro configuraciones con N = 10 000

5.3. Comparación de configuraciones

Tabla 4. Pruebas de uniformidad por configuración, N = 10 000 (decisión de ambas pruebas sobre H0)

Generador	χ^2	p_{χ^2}	D	p_{KS}	Decisión
glibc	2.44	0.983	0.0071	0.687	No se rechaza
APE 5	14.16	0.117	0.0060	0.869	No se rechaza
RANDU	8.81	0.455	0.0064	0.813	No se rechaza
Módulo pequeño	937.50	< 0.001	0.0625	< 0.001	Se rechaza

Los tres generadores de módulo grande pasan ambas pruebas. El de módulo pequeño las falla con claridad: sus 16 valores posibles reparten los datos de forma desigual entre los diez subintervalos, y su distribución acumulada avanza a saltos de 1/16. RANDU pasa ambas pruebas aunque es un generador defectuoso: sus tríos de valores consecutivos caen sobre unos pocos planos paralelos [3]. Esa falla no aparece en pruebas que miran un valor a la vez.

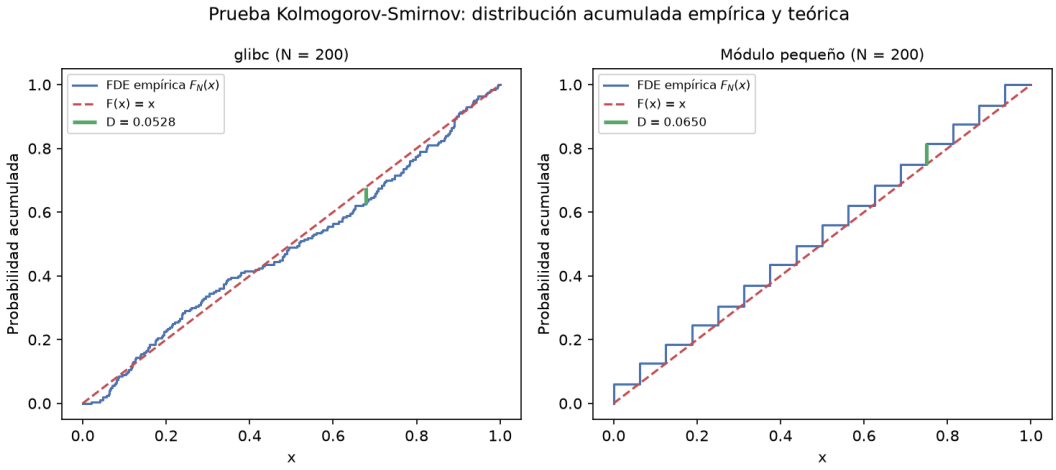


Figura 2: Distribución acumulada empírica frente a F(x) = x con N = 200, marcando la distancia máxima D

La figura usa N = 200 para que se vean los escalones. Con ese tamaño, Kolmogorov-Smirnov no detecta el módulo pequeño: D = 0.065 queda por debajo del valor crítico $1.36/\sqrt{200} \approx 0.096$ (p = 0.356). Chi-cuadrado sí lo rechaza ($\chi^2 = 19.0$, p = 0.025), porque los 16 valores posibles dejan subintervalos con 24 a 26 datos y otros con 12 o 13, frente a 20 esperados. Con N = 10 000 el valor crítico de Kolmogorov-Smirnov baja a 0.0136 y ambas pruebas lo rechazan: una muestra más grande le da más poder a la prueba.

5.4. Python frente a Java

Las dos implementaciones producen la misma secuencia: 0 diferencias en los 10 000 valores de cada una de las cuatro configuraciones. Los estadísticos y los valores p también coinciden en todos los casos.

La tabla de resultados esperados de la guía pide las columnas π estimado y Error, que ningún paso de esta práctica produce, porque corresponden a una estimación por Monte Carlo. Siguiendo la tabla de la diapositiva 18 [2], se reemplazaron por los estadísticos y valores p de ambas pruebas. Se mantienen N y los tiempos de Python y de Java.

Tabla 5. Generador glibc según el tamaño de muestra

N	χ^2	p_{χ^2}	D	p_{KS}	**Tiempo Python (s)**	**Tiempo Java (s)**
1 000	10.06	0.346	0.02839	0.391	0.0007	0.0007
10 000	2.44	0.983	0.00713	0.687	0.0070	0.0047
100 000	9.52	0.391	0.00313	0.279	0.0837	0.0153
1 000 000	8.85	0.452	0.00068	0.742	1.1558	0.1362

En ningún tamaño se rechaza H0. D baja a medida que crece N, como se espera de una secuencia uniforme, mientras que los valores p varían sin tendencia, porque bajo H0 se distribuyen de forma uniforme. Con N = 1 000 los tiempos son parecidos; con N = 10 000 Java ya es 1.5 veces más rápido, y con 1 000 000 de números, unas 8 veces.

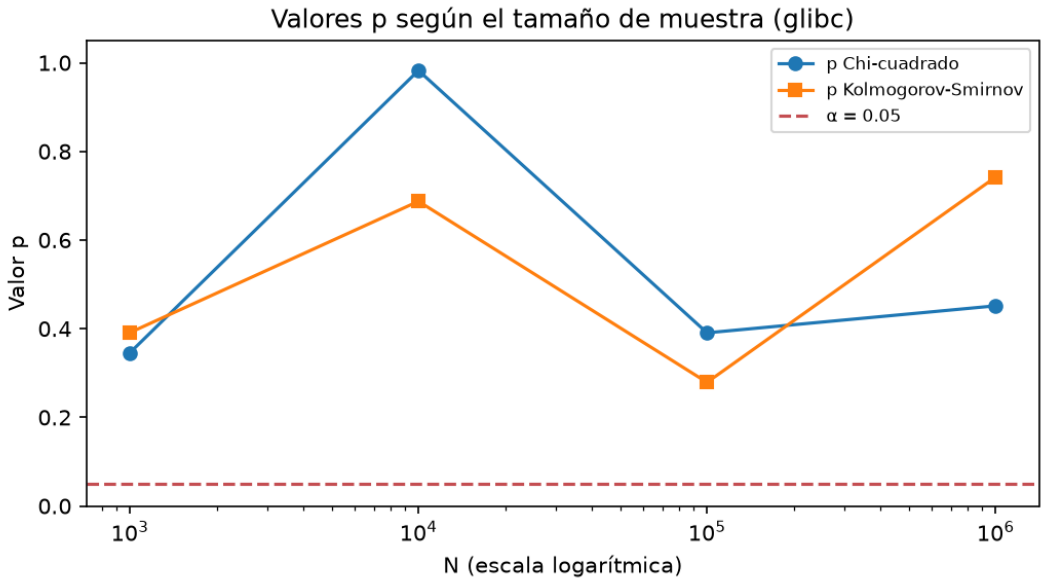


Figura 3: Valores p de Chi-cuadrado y Kolmogorov-Smirnov según N para el generador glibc

6. Preguntas de Control

Las respuestas se basan en las diapositivas de la semana 6 [2], en el capítulo de generación de números aleatorios de Banks et al. [4] y en los resultados de esta práctica.

1. ¿Qué es la hipótesis nula? Es la afirmación que se pone a prueba y que se mantiene mientras los datos no la contradigan [4]. En esta práctica, H0 dice que la secuencia proviene de una distribución uniforme U(0, 1) [2].

2. ¿Qué representa el valor p? Es la probabilidad, bajo H0, de observar un estadístico al menos tan extremo como el obtenido [2]. Un p pequeño indica que los datos serían raros si H0 fuera cierta. Con glibc y N = 10 000, p = 0.983 en Chi-cuadrado.

3. ¿Qué significa rechazar H0? Que los datos son demasiado improbables bajo la uniformidad ($p < \alpha$) y hay evidencia de que el generador no produce valores uniformes, como pasó con el módulo pequeño. Se acepta un riesgo $\alpha = 0.05$ de rechazar un generador que sí es uniforme, el error tipo I [2].

4. ¿Qué diferencia existe entre Chi-cuadrado y KS? Chi-cuadrado agrupa los datos en subintervalos y compara frecuencias, así que su resultado depende de cuántos subintervalos se elijan. Kolmogorov-Smirnov trabaja con cada valor y compara las distribuciones acumuladas [4]. Las diapositivas resumen la regla práctica: Chi-cuadrado para datos agrupados o discretos y Kolmogorov-Smirnov para datos continuos [2]. Con N = 200, sin embargo, Chi-cuadrado detectó el generador de módulo pequeño y Kolmogorov-Smirnov no.

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio? Porque una prueba solo detecta patrones específicos de no aleatoriedad, y no rechazar H0 no significa que H0 sea verdadera, solo

que no hay evidencia suficiente contra ella [2]. RANDU pasó Chi-cuadrado y Kolmogorov-Smirnov y aun así sus tríos de valores caen en pocos planos [3]. Además, un LCG es determinista: con la misma semilla repite siempre la misma secuencia.

7. Conclusiones

- Los programas en Python y en Java dieron exactamente los mismos números y los mismos resultados, así que las dos implementaciones del generador son equivalentes. Además, coinciden con el ejemplo visto en clase, salvo una diferencia mínima en un valor p.
- Las pruebas Chi-cuadrado y Kolmogorov-Smirnov sirvieron para separar los generadores que pasan de los que fallan: los tres de módulo grande pasaron y el de módulo pequeño fue rechazado por ambas pruebas.
- Un valor p alto no prueba que un generador sea aleatorio; solo indica que no se encontró evidencia en su contra. RANDU lo deja claro: pasa ambas pruebas y aun así es defectuoso.
- La cantidad de datos importa. Con pocos números, un generador malo puede pasar desapercibido para una prueba; con más datos, las dos lo detectaron.
- Java fue unas 8 veces más rápido que Python con un millón de números, mientras que con mil números los tiempos fueron casi iguales.

8. Recomendaciones

- Complementar Chi-cuadrado y Kolmogorov-Smirnov con pruebas de independencia, como la prueba de rachas, antes de usar un generador nuevo.
- Usar muestras de al menos 10 000 números para que las pruebas detecten desviaciones pequeñas.

Bibliografía / Referencias

[1] W. H. Press, S. A. Teukolsky, W. T. Vetterling y B. P. Flannery, *Numerical Recipes 3rd Edition: The Art of Scientific Computing*, 3.ª ed. Cambridge: Cambridge University Press, 2007, ISBN: 9780521880688.

[2] J. O. Guamán, *Pruebas estadísticas de uniformidad e independencia*, Diapositivas de clase, semana 6, asignatura Simulación, Carrera de Computación, Universidad Nacional de Loja, Loja, Ecuador, 2026.

[3] G. Marsaglia, «Random Numbers Fall Mainly in the Planes,» *Proceedings of the National Academy of Sciences*, vol. 61, n.º 1, págs. 25-28, 1968. DOI: 10.1073/pnas.61.1.25

[4] J. Banks, J. S. Carson, B. L. Nelson y D. M. Nicol, *Discrete-Event System Simulation*, 5.ª ed. Upper Saddle River, NJ: Prentice Hall, 2010, ISBN: 9780136062127.

9. Anexos

El código de Python (paquete simulacion_ape, con sus pruebas en pytest) y el de Java (paquete ape6, con modelo, vista y controlador) están en la carpeta codigo del repositorio de la asignatura, junto con los resultados en CSV y las figuras.

Declaración de uso de IA Generativa

Se usaron herramientas de IA generativa como asistente durante la práctica. Las tablas siguientes resumen las herramientas y las instrucciones dadas.

Tabla A. Herramientas utilizadas

Herramienta	Uso
el Gentleman (entorno Pi), con un modelo Claude de Anthropic	Leer la guía y las diapositivas, escribir el código, generar las gráficas y redactar y maquetar el informe
Python 3 (Matplotlib, SciPy, pytest)	Implementación del LCG y de las pruebas, gráficas y verificación
Java/JDK 21	Implementación del mismo algoritmo y de las mismas pruebas
LaTeX	Maquetación del documento

Tabla B. Prompts por iteración (resúmenes de las instrucciones, no texto literal)

Iteración	Prompt (resumen)
1. Lectura de la guía	Lee la guía de la APE 6 y explica qué pide la práctica y por qué la tabla de resultados pide un π estimado.
2. Revisión de las diapositivas	Revisa las diapositivas de la semana 6 y comprueba si se estima π o qué tabla de resultados usa el docente.
3. Implementación	Implementa el LCG y las pruebas Chi-cuadrado y Kolmogorov-Smirnov en Python y en Java con los mismos parámetros.
4. Comparación	Compara configuraciones del generador y tamaños de muestra de 1 000 a 1 000 000, con estadísticos, valores p y tiempos de ambos programas.
5. Informe	Redacta el informe con los resultados, las preguntas de control y las conclusiones.
6. Estructura de Java	Explica por qué Java no usa MVC; el docente prefiere el código dividido, así que divídelo igual que Python.
7. Revisión	Corrige lo que señaló la verificación: muestra la secuencia generada y revisa la afirmación sobre N = 200.
8. Citas y conclusiones	Cita de dónde salen las respuestas de las preguntas de control, haz las conclusiones menos técnicas e incluye la declaración de IA.

Todo el código, los resultados y el texto fueron revisados, ejecutados y validados por el autor, quien asume la responsabilidad sobre la calidad y veracidad del trabajo.

Alejandro Padilla
Autor